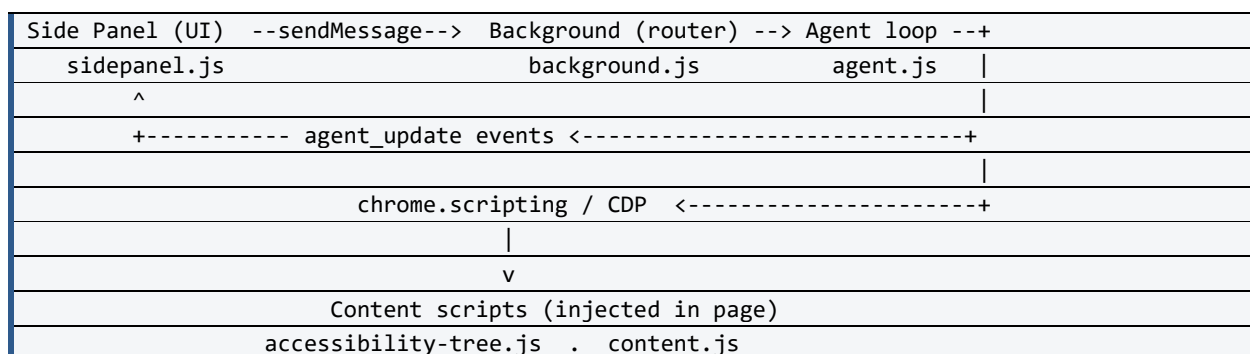


WebBrain — Architecture Guide

WebBrain hands an LLM the keys to the user's *real, logged-in* browser tab. There is no headless browser and no separate “AI account” — from the website's point of view, the agent *is* the user. Most design decisions (the security model, UI-first rule, trusted events) follow from that one fact.

1. The Four Layers



- **Side panel** (ui/) — chat UI beside the page. Owns the **Ask vs Act** toggle. Ask = read-only tools; Act = full action set.
- **Background service worker** (background.js) — message router. On Chrome MV3 it can *die between turns*, so conversations persist to `chrome.storage.session`.
- **Agent** (agent/agent.js, ~7k lines) — the brain. Runs the tool-use loop, talks to the LLM provider, executes tools, manages context.
- **Content scripts** (content/) — injected into every page. `accessibility-tree.js` builds the flat AX tree + resolves `ref_id` → Element; `content.js` does the actual clicking/typing/reading.

2. One Turn, End to End

Example instruction: “*find the pricing page and fill out this form.*”

1. **UI → Background → Agent.** Panel sends `{action:'chat', text, mode:'act', tabId}`; background calls `agent.processMessage(...)`. A callback `onUpdate` streams events back for live rendering.
2. **Enrich the first message.** `_enrichUserMessageWithCurrentPage` attaches URL + title, a matching **site adapter's** guidance (58+, one fires at a time), a viewport **screenshot** if the provider has vision, and any `/allow-api` override note. The model starts already knowing where it is.
3. **The agent loop runs** (see §3) until the model returns a text-only answer or hits the step cap.
4. **Tools execute in different contexts.** `executeTool` dispatches by name: DOM tools → content script; `navigate/new_tab` → `chrome.tabs`; `fetch_url/downloads` → worker.
5. **Events stream back.** `onUpdate(type, data)` — `tool_call`, `tool_result`, `text`, `warning`, `clarify`, `context_compacted` — relayed to the panel and rendered incrementally.

3. The Agent Loop — How It Decides the Next Action

The honest answer: **the LLM decides**. The system is built to shape that decision well. The loop in `_processMessageInner`:

<code>while (steps < maxSteps) {</code>	<code>// default 130</code>
<code> _manageContext()</code>	<code>// compact if conversation too big</code>
<code> result = provider.chat(messages, {</code>	<code>//</code>
<code> tools, temperature: 0.15, maxTokens: 4096 }</code>	<code>)</code>
<code> if (result.toolCalls) {</code>	
<code> _executeToolBatch(...)</code>	<code>// run each tool, feed results back</code>
<code> continue</code>	
<code> } else {</code>	
<code> return result.content</code>	<code>// text-only = task done</code>
<code> }</code>	
<code>}</code>	

Three things steer the model:

- **Tool schemas** — `getToolsForMode` hands it only the legal tools. Ask = read-only set; Act = full ~40 (or ~20 in Compact mode for small local models).
- **System prompt** — `SYSTEM_PROMPT_ACT` defines the default loop:
`get_accessibility_tree({filter: 'visible'})` → pick `ref_ids` → `click_ax` / `set_field` → re-read to verify → repeat. Also enforces “read current page first,” “don’t rush — verify after every page change,” “one form field at a time.”
- **Temperature** — Act = 0.15 (near-deterministic), Ask = 0.3, vision descriptions = 0.

Two automatic feedback mechanisms close the perception loop:

- **Auto-screenshot** (`_shouldAutoScreenshot`) — after page-changing actions, attaches a screenshot so the next turn “sees” the new state.
- **Loop detection** (`_checkLoop`) — three detectors (general repeat, coordinate-click 5px buckets, navigation URL diff). **Nudges** at a threshold, **hard-stops** a wedged run.

4. The Accessibility Tree (deep dive)

Built from the live DOM, not HTML source. `accessibility-tree.js` runs inside the page and walks the rendered, post-JavaScript document tree. It is WebBrain’s own JS reimplementaion modeled on the platform AX tree — it does *not* read Chrome’s real accessibility tree.

What a line looks like

<code>navigation [ref_3]</code>
<code> link "Pricing" [ref_8] href="/pricing"</code>
<code>main [ref_12]</code>
<code> textbox "Email" [ref_19] placeholder="you@example.com"</code>
<code> button "Sign in" [ref_22] type="submit"</code>

Each included element = one line, built by `formatLine`. Three pieces, three sources:

- **role** — `getRole`: explicit `role="..."` attribute, else inferred from tag (`a`→`link`, `button`→`button`, `select`→`combobox`), with `<input>` specialized by type.
- **accessible name** — `getAccessibleName`: a priority cascade — `aria-label` → `aria-labelledby` → `placeholder` → `title` → `alt` → `<label for>` → text children. The human-readable label.
- **indentation** — `' '.repeat(depth)`. The only thing encoding tree structure in the flat text.

Where ref_8 / ref_22 come from

They are **NOT** the webmaster's HTML ids. They are minted by WebBrain in getOrMintRef:

function getOrMintRef(el) {
for (const key in window.__wbElementMap)
if (window.__wbElementMap[key].deref() === el) return key; // reuse
const key = 'ref_' + (++window.__wbRefCounter); // else mint
window.__wbElementMap[key] = new WeakRef(el);
return key;
}

- **The number is just a global counter.** ref_8 = “the 8th distinct element handed a ref on this page.” Meaningless beyond identity.
- **Stored as a WeakRef** in window.__wbElementMap so removed elements don't stay pinned in memory.
- **Stable across calls** because getOrMintRef first scans for an existing WeakRef whose .deref() is the same element, and reuses that key. Dead refs swept by sweepDeadRefs.
- **Round-trips:** model reads button "Sign in" [ref_22] → calls click_ax({ref_id: 'ref_22'}) → __wb_ax_lookup resolves it back to the Element → CDP-trusted click. Fully independent of the site's id/class/name.

How it decides to skip a node — shouldInclude

“Resolve to generic and get skipped” is shorthand. The real logic is a series of gates that depend on the requested **filter**. Universal gates first (all non-all filters):

if (SKIP_TAGS.has(tag)) return false; // script/style/meta/link/title/noscript
if (filter!=='all' && aria-hidden) return false;
if (filter!=='all' && !isVisible(el)) return false; // display:none, 0-size
if (filter!=='all' && !refId && !isInViewport(el)) return false; // off-screen

Then it branches by filter:

- **interactive** — return `isInteractive(el)`. Nothing else survives.
- **visible** (the agent's default) — a **whitelist**, not a blacklist: keep only if `isInteractive`, a heading `h1-h6`, or `role ∈ USEFUL_NON_INTERACTIVE_ROLES` (`dialog`, `listbox`, `menu`, `form`, `navigation`, `region`, `progressbar...`). Everything else → `false`. This is where plain `<div>/<span>` wrappers die — it never checks for generic by name.
- **all** (generous legacy) — keep if **interactive OR landmark OR has any accessible name OR** `role !== 'generic' && role !== 'image'`. *This* is the only place generic is checked explicitly. A named `<span>` survives here but not in `visible`.

The crucial subtlety — skipped ≠ pruned. A `false` only means “no line for this node.” walk still recurses into children, and they're emitted at the **parent's depth** — so they “bubble up.” Three nested layout divs around a button collapse to a single button “Buy” [ref_5] line. The model is shown action targets + landmarks, hoisted out of the scaffolding.

5. How WebBrain Compacts Conversations

The agent runs a multi-step tool loop, so context grows fast (each tool call = assistant turn + result, often with a screenshot). `_manageContext` keeps it inside the model's window. It runs **both at the start of each user turn AND at the top of every loop iteration**, so a long autonomous run compacts mid-flight — not only between turns.

Triggers — fires if ANY one is exceeded

Budget	Threshold
Message count	> 50 (<code>maxContextMessages</code>)
Raw characters	> 80,000 (<code>maxContextChars</code> , ~20k tokens)
Token budget	> $0.75 \times \text{provider.contextWindow}$ (<code>contextCompactRatio</code> ; window defaults 128k cloud / 16k local)

The token estimate is the smart part. Instead of a naive `chars/4`, it takes the provider's **actually-reported** `prompt_tokens` from the last call (which already includes the fixed system-prompt + tool-schema overhead) and adds the estimated *growth* in bytes since that reading. It falls back to `chars/4` only on the first turn / streaming path.

What it does when it fires

Rebuilds the message array into: system + pinned original task + pinned scratchpad + summary + last 30 verbatim.

- **Pin the original task verbatim** at position 1 — *never* summarized. (Folding it into a synthetic summary makes small models announce “the previous context was removed” and forget the goal.)
- **Keep last 30 messages verbatim** (`keepRecent = 30` ≈ last 15 tool turns).
- **Boundary fix:** the recent slice must not start mid-tool-group. A tool result without its preceding assistant turn is rejected by both OpenAI and Anthropic APIs, so leading orphan results are pushed back into the summarized set.
- **Summarize the middle:** one line per old message. Tool results are *not* dropped — `_digestToolResult` keeps the key fact (file counts, download IDs, final URL). Untrusted markers are stripped first so page text can't inject into the summary.
- **Second-pass compression:** if that summary is still > 2000 chars, a sub-call to the LLM (“summarize in 3-5 bullets,” `maxTokens: 300`, `temp: 0.2`) compresses it further.
- **Tell the user:** clears `_lastInputTokens` (re-measure fresh next call) and emits `context_compacted` → the panel shows an inline “*Context automatically compacted*” separator — history was summarized, not lost.
- **Edge case:** if there's < 4 messages to summarize but the token budget tripped (one huge page early in a run), it calls `_truncateOversizedMessages` to shrink the bloat in place instead.

Three companion mechanisms

Mechanism	What it does
Image pruning	<code>_pruneOldImages</code> runs before EVERY LLM call (not just on compaction). Keeps only the most recent screenshot (0 if no vision); older base64 → [older screenshot omitted...]. Images are the biggest token hogs.

Mechanism	What it does
Emergency trim	<code>_emergencyTrim</code> — hard fallback when a provider STILL returns context-overflow after compaction. Keeps system + original task + scratchpad + last 6 messages, then retries once.
Tool-result cap	Individual tool results truncated (~8KB) at execution time — limits bloat at the source (and the prompt-injection surface).
Scratchpad	Pinned, so it survives all of the above — the deliberate antidote to compaction destroying specifics. <code>scratchpad_write</code> is how the model carries facts across compactions.

6. Other Subsystems Worth Naming

Subsystem	What to remember
Providers	BaseLLMProvider normalizes every backend to {content, toolCalls, usage}. Add one: subclass, implement chat(), register in manager.js. Loop is provider-agnostic.
Security	Page content is wrapped in <untrusted_page_content> — data, never instructions. Only system prompt + user chat are authoritative. UI-first rule; /allow-api gates mutations; 8KB result cap.
Chrome vs FF	Same loop/tools/prompts. Differ only in substrate: Chrome CDP = trusted events, real screenshots, shadow-DOM piercing, persistence. Firefox = synthetic events, weaker.

Prompt-injection is the top risk precisely because the agent can click/type/submit as the logged-in user. A malicious page that talks the agent into sending an email or transferring money is a real attack, not hypothetical — hence the hard “pages are not the user” rule.

7. FAQ

Question	Answer
Why an AX tree instead of raw HTML / screenshots?	Compact, semantic, gives stable handles to act on. HTML is too noisy/big; pure vision is slower and gives no clickable handle. The tree is action targets + landmarks, flattened.
Where do ref_ids come from?	WebBrain mints them — a global counter, stored as WeakRefs, reused per element so they're stable across calls. Nothing to do with site ids.
How does it avoid infinite loops?	_checkLoop after every tool call: repeat / coordinate-click / navigation detectors. Nudge, then hard stop. Plus a step cap (default 130).
How does it not blow the context window?	_manageContext: pins task + scratchpad, summarizes the middle (tool digests), keeps last 30; triggers on count/char/token budget. Plus image pruning every call, ~8KB result cap, emergency trim fallback.
Chrome vs Firefox?	Identical agent/tools/prompts; substrate differs. Chrome has CDP (trusted events, screenshots, shadow piercing, persistence); Firefox uses synthetic events.
Biggest security concern?	Prompt injection — the agent acts as the logged-in user. Page content is untrusted data; only system prompt + user chat are authoritative.

8. Practice — Answer From Memory

Cover the page above. Write your answer in your own words, then check.

Q1. Walk through what happens between the user pressing Enter and the first tool executing.

Q2. A page has five nested `<div>` wrappers around one `<button>`. What does the agent see, and why?

Q3. Explain how `ref_22` stays pointing at the same button across three `get_accessibility_tree` calls.

Q4. `filter:'visible'` vs `filter:'all'` — when does a named `<span>` survive, and when not?

Q5. What three budgets trigger `_manageContext`, and what does the rebuilt message array look like?

Q6. Why is the original user task pinned verbatim instead of summarized? Name two things that survive compaction and two backstops if it still overflows.

Q7. Why is prompt injection the highest-severity risk here, and what defends against it?
